CS-107 Lecture 4
Binary Arithmetic

Troy Astarte

Megan Venn-Wycherley

Recap: why we convert number bases

- **Human readability:** long strings of digits are hard to read and keep track of. Hex and octal are compact shorthand for binary.
- **Programming and input/output:** low-level work such as bit masks needs base 10 data in another base.
- **Two directions to know:** base b to decimal, and decimal to base b.
  - **Between two bases:** go through decimal!

Recap: base b to decimal

- **The formula:** convert each digit d_i to decimal, multiply it by b^i, then sum the results.
- **Octal example:** $(357)_8 = 3 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0 = 192 + 40 + 7 = (239)_{10}$.
- **Hex example:** $(AE4)_{16} = 10 \cdot 256 + 14 \cdot 16 + 4 \cdot 1 = 2560 + 224 + 4 = (2788)_{10}$.
- **Letters are digits:** A = 10 up to F = 15, so $(AE4)_{16}$ is a three-digit number.

Recap: decimal to base b

- **The algorithm:** while the number is not zero, divide by the new base b, note the remainder, replace the number by the quotient.
- **Read the remainders in reverse** order to get the value in base b.
- **To base 8:** 239 / 8 = 29 remainder 7, 29 / 8 = 3 remainder 5, 3 / 8 = 0 remainder 3, so $(357)_8$.
- **To base 16:** 239 / 16 = 14 remainder 15 (F), 14 / 16 = 0 remainder 14 (E), so $(EF)_{16}$.

Recap: base x to base y

- **Same algorithm,** but the division now happens in base x, which is not intuitive to us.
- **So go via decimal:** base x to decimal, then decimal to base y.
- **Worked example:** $(A6C)_{14}$ = 1960 + 84 + 12 = $(2056)_{10}$, and 2056 in base 12 is $(1234)_{12}$.
- **Powers of 2 need no arithmetic:** group binary digits from the right, threes for octal and fours for hex.

Goals for Today

- Arithmetic in binary
- Importance of binary in computing
- Looking ahead to using number representations

Binary Addition

Binary arithmetic - Addition

- Arithmetic can be done directly on binary numbers as long as we remember the suitable addition and multiplication tables.

- In particular we must remember that **1 + 1 = 10** in binary. This gives a **carry.**

Why?

Binary Addition Table		
+	**0**	**1**
0	0	1
1	1	10
Binary Multiplication Table		
·	**0**	**1**
0	0	0
1	0	1

Decimal Addition

- Example:

$$\begin{array}{r} 87_{10} \\ +\ 75_{10} \\ \hline \end{array}$$

Decimal Addition

- Example:

$$\begin{array}{r} 87_{10} \\ +\ 75_{10} \\ \hline 2 \end{array}$$

Decimal Addition

- Example:

$$\begin{array}{r} {\scriptstyle 1} \\ 87_{10} \\ +\ 75_{10} \\ \hline 2 \end{array}$$

Decimal Addition

- Example:

$$\begin{array}{r} {\scriptstyle 11} \\ 87_{10} \\ +\ 75_{10} \\ \hline 62 \end{array}$$

Decimal Addition

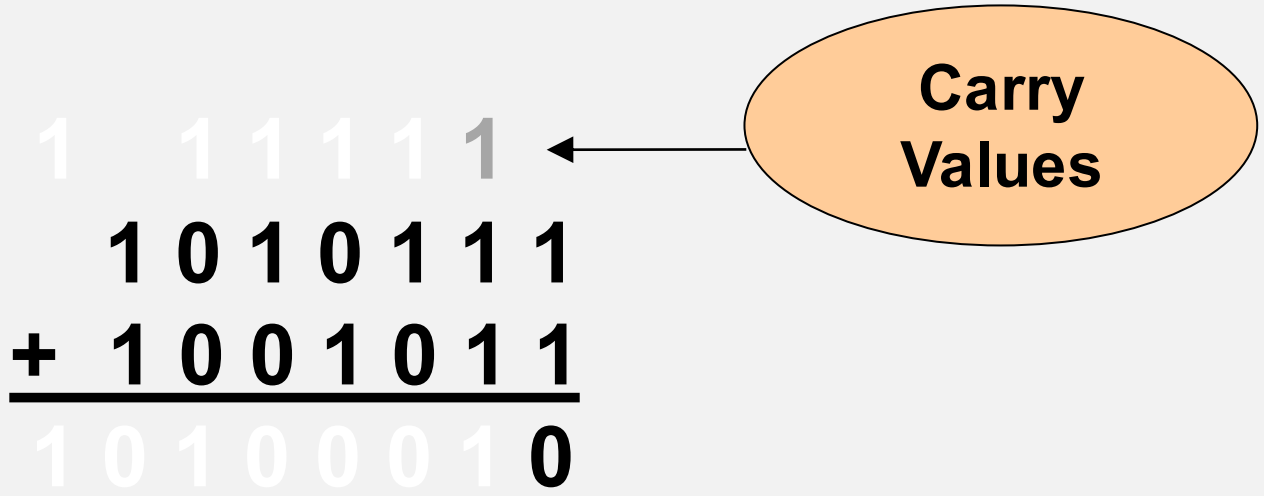
- Example:

$$\begin{array}{r} {\scriptstyle 11} \\ 87_{10} \\ +\ 75_{10} \\ \hline 162 \end{array}$$

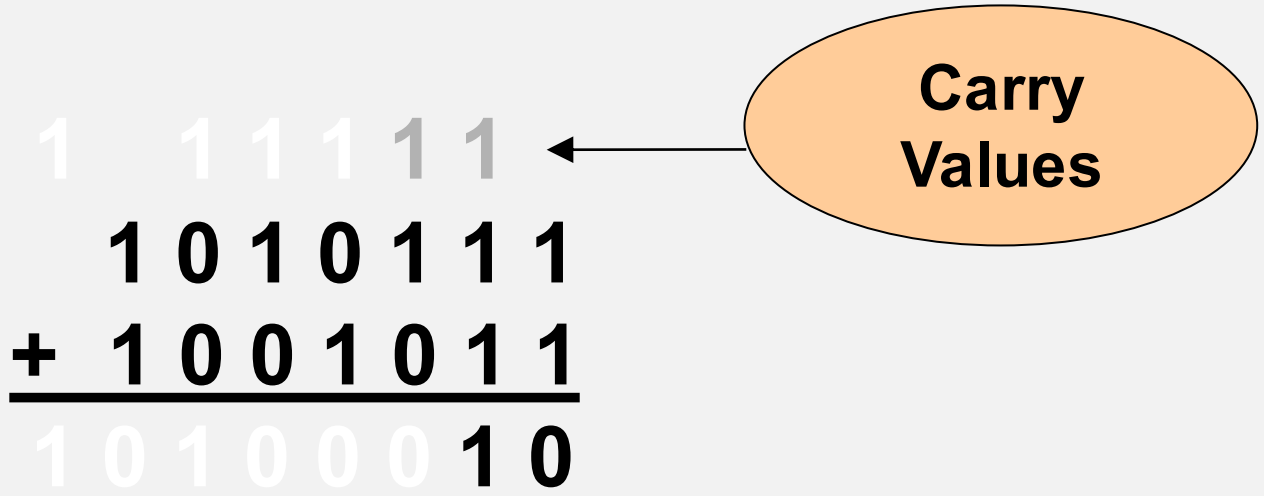
Binary arithmetic - Addition

1 0 1 0 1 1 1
+ 1 0 0 1 0 1 1

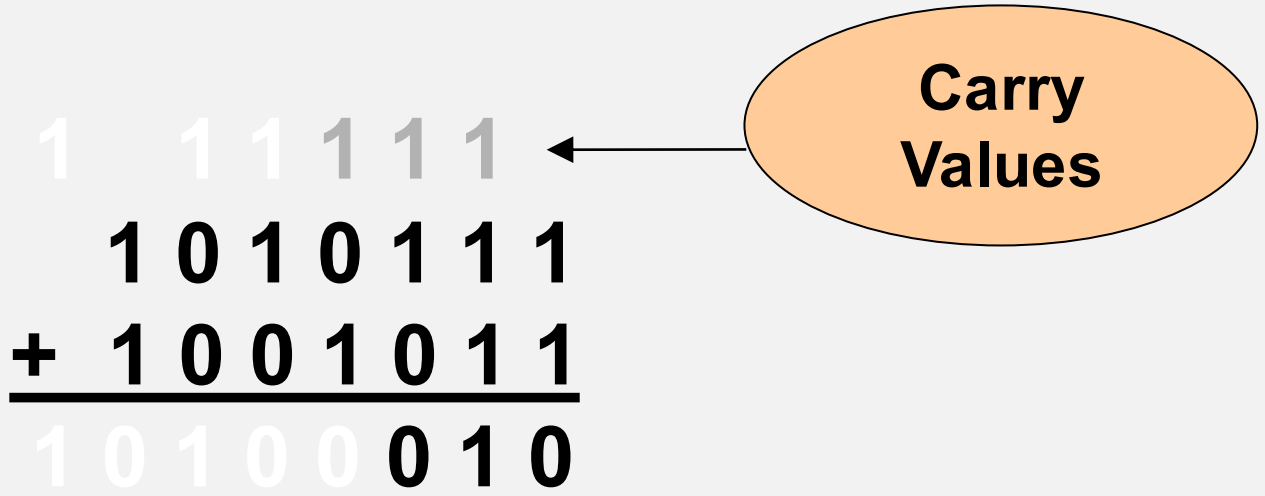
Binary arithmetic - Addition



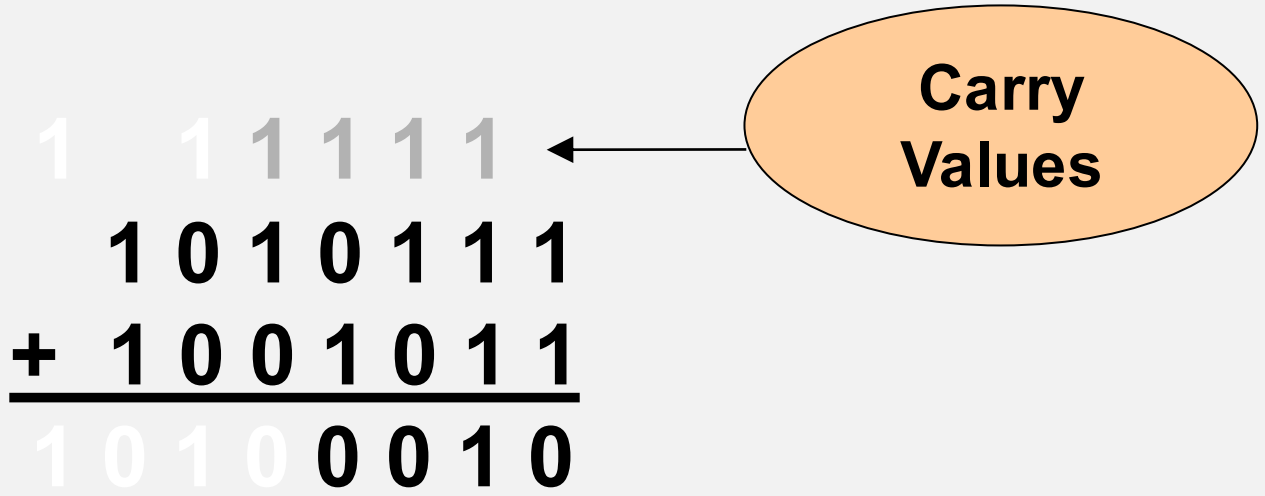
Binary arithmetic - Addition



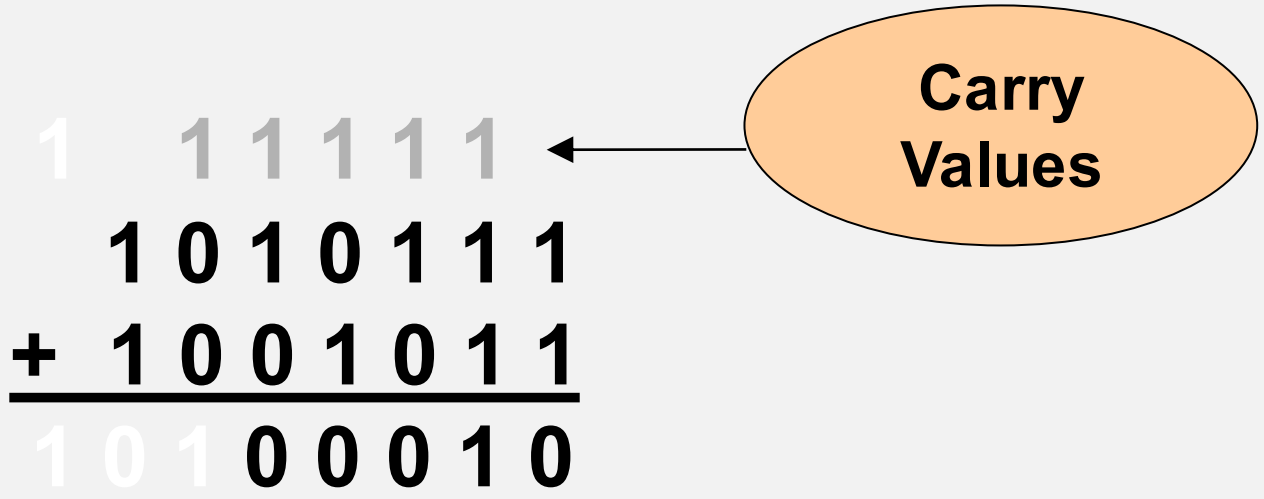
Binary arithmetic - Addition



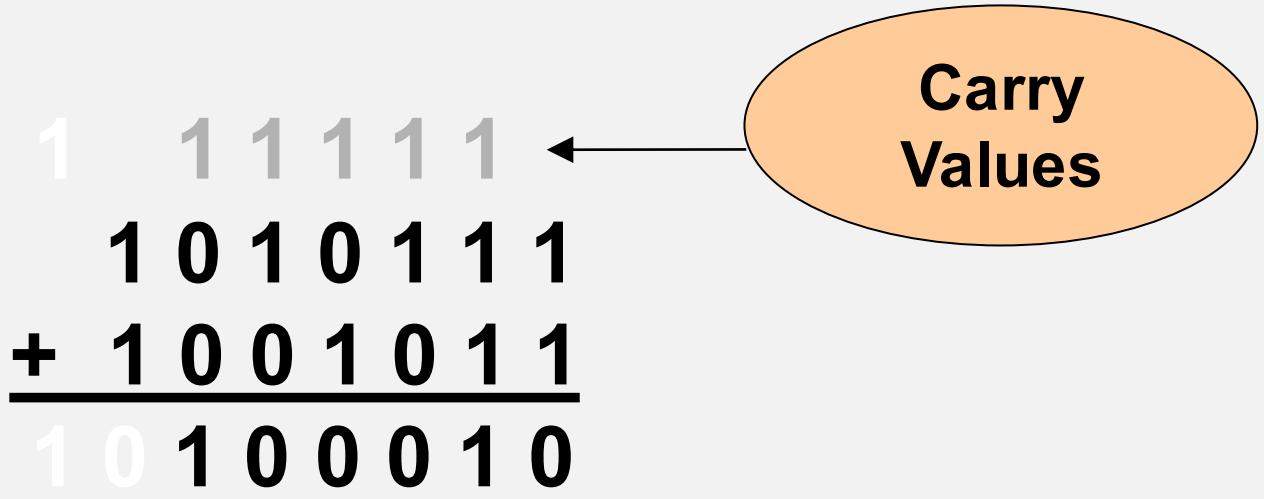
Binary arithmetic - Addition



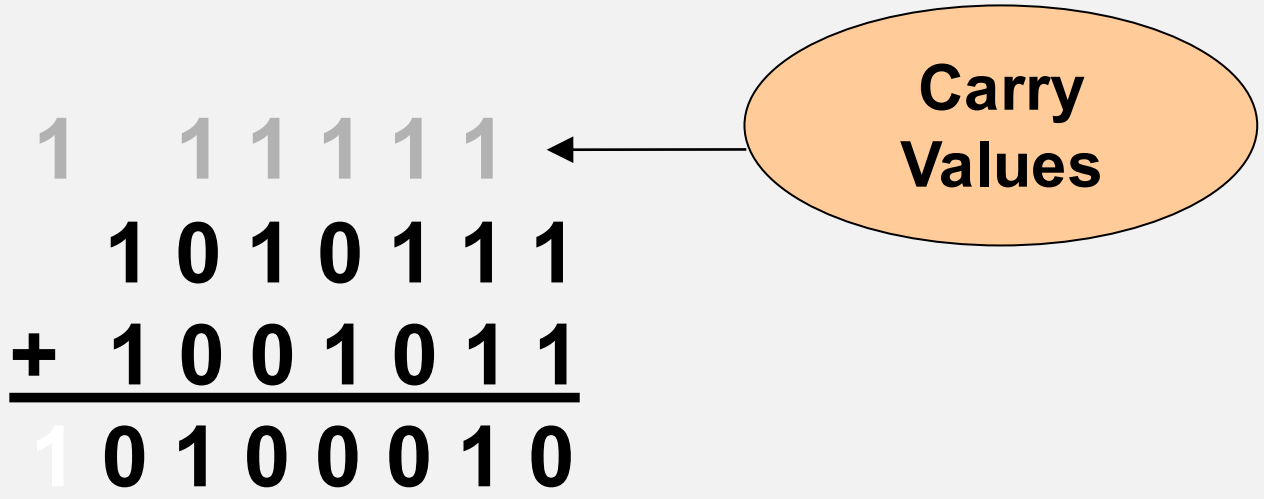
Binary arithmetic - Addition



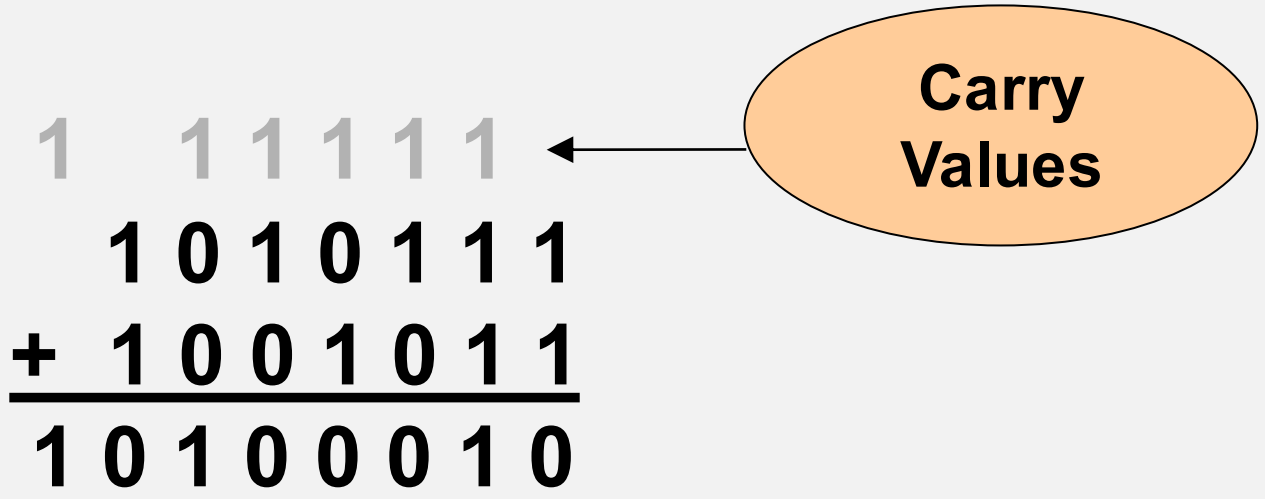
Binary arithmetic - Addition



Binary arithmetic - Addition



Binary arithmetic - Addition



Try it out

1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1

Try it out

```
          1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
---------------
          0
```

Try it out

```
        1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
---------------
      1 0
```

Try it out

```
      1 1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
---------------
    1 1 0
```

Try it out

```
    1 1 1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
-----------------
    1 1 1 0
```

Try it out

```
  1 1 1 1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
-----------------
    0 1 1 1 0
```

Try it out

```
  1 1 1 1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
-----------------
  1 0 1 1 1 0
```

Try it out

```
  1 1 1 1 1
  1 0 1 0 1 0 1
+ 0 0 0 1 1 1 1
-----------------
  1 1 0 1 1 1 0
```

Binary Subtraction

Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.

$$\begin{array}{rr} & 87_{10} \\ - & 59_{10} \\ \hline \end{array}$$

Binary arithmetic - Subtraction

For subtraction we have a concept of “**borrowing**” from a higher position.

$$
\begin{array}{rr}
 & 1 \\
 & 87_{10} \\
- & 59_{10} \\
\hline
 & 8
\end{array}
$$

Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.

$$
\begin{array}{rr}
 & \mathbf{1} \\
 & \cancel{8}77_{10} \\
- & \ \ 59_{10} \\
\hline
 & 28
\end{array}
$$

Binary arithmetic - Subtraction

For subtraction we have a concept of “**borrowing**” from a higher position.

```
    1  0  1  0  1  1  1
-      1  1  1  0  1  1
-------------------------
```

Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.

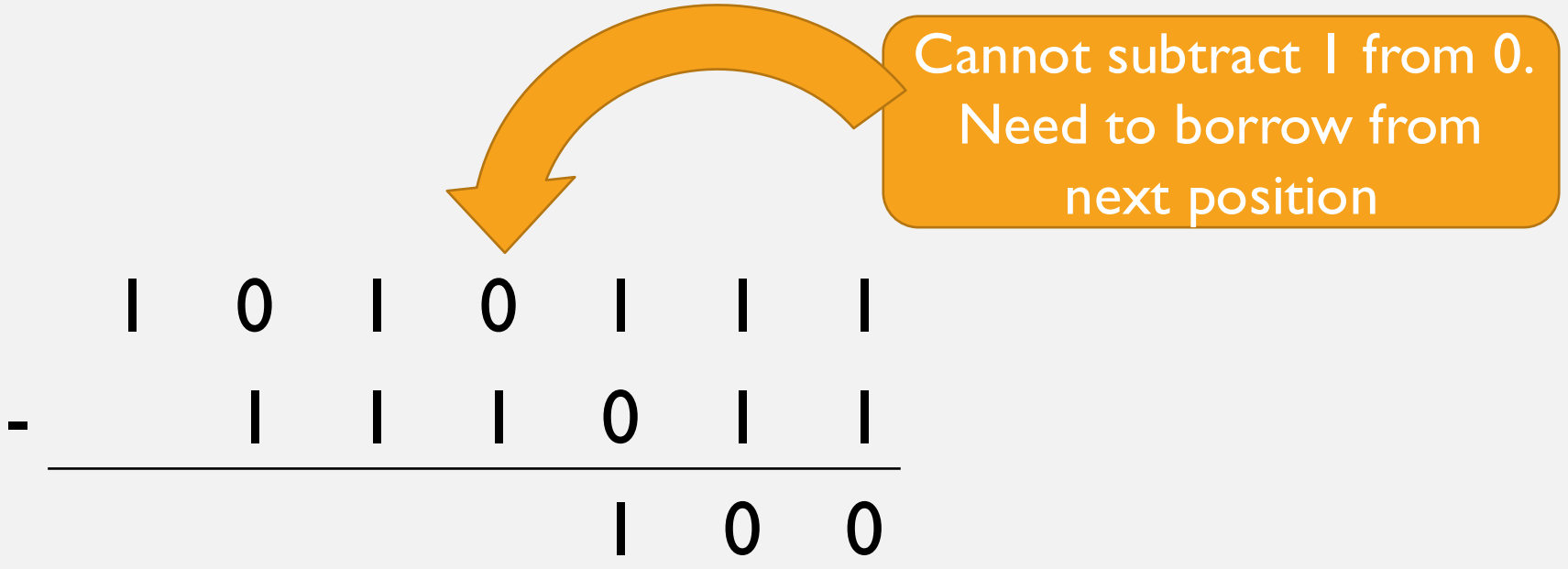
```
    1  0  1  0  1  1  1
-      1  1  1  0  1  1
-------------------------
                      0
```

Binary arithmetic - Subtraction

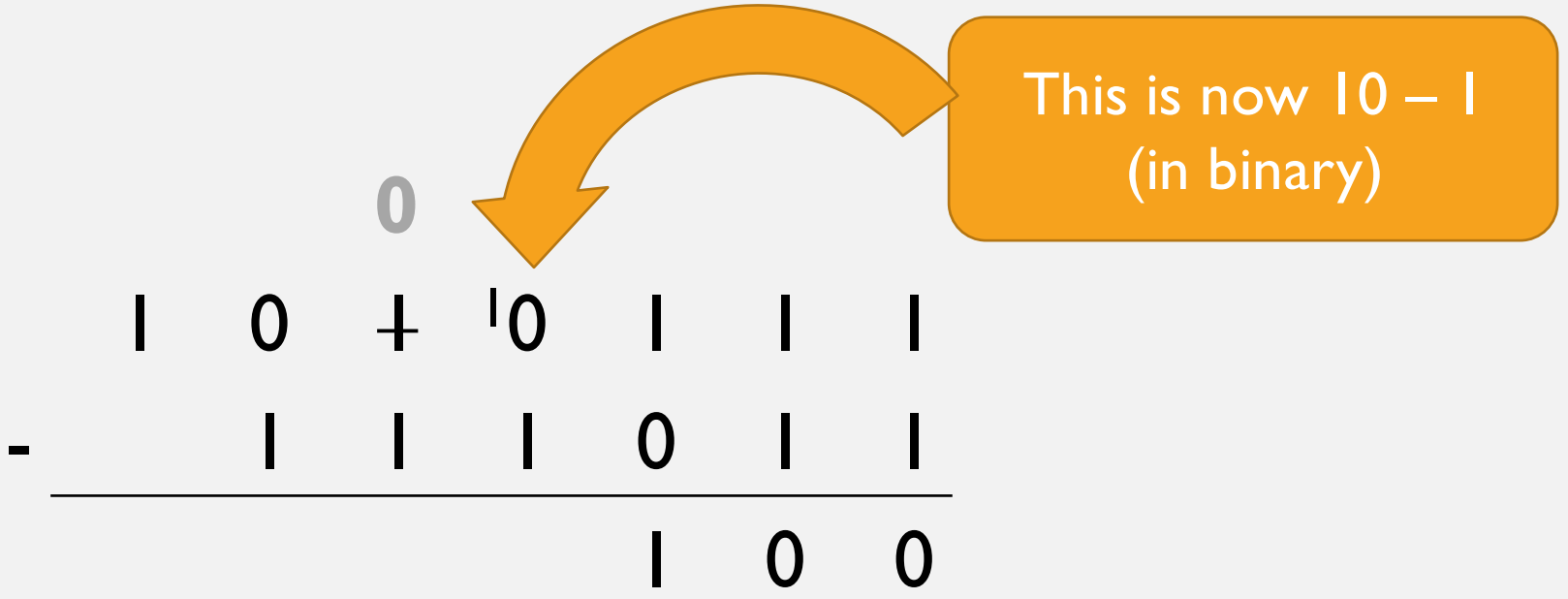
For subtraction we have a concept of "**borrowing**" from a higher position.

```
    1  0  1  0  1  1  1
-      1  1  1  0  1  1
-------------------------
                   0  0
```

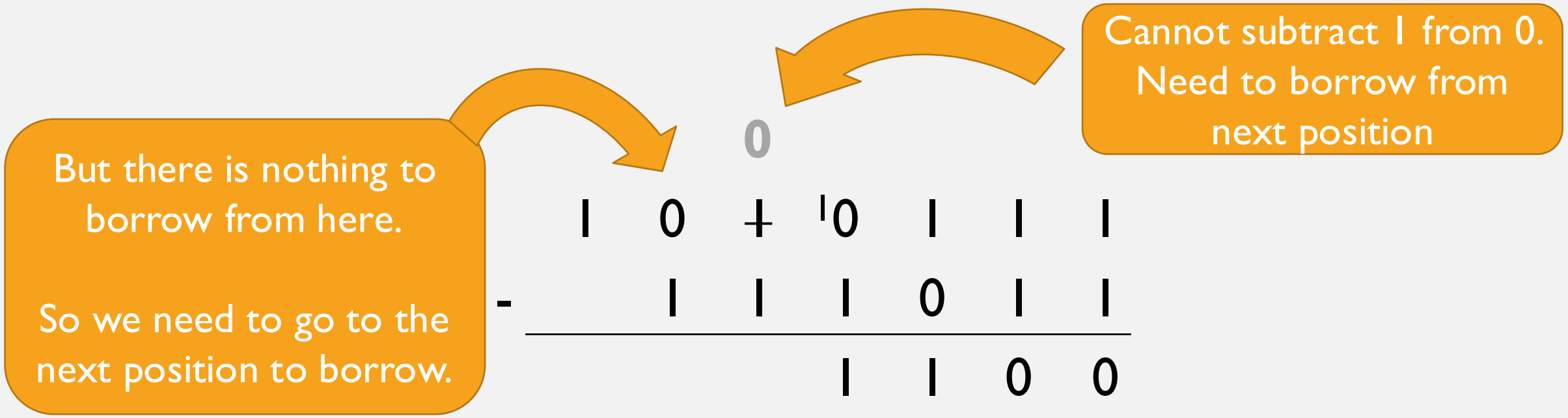
Binary arithmetic - Subtraction



Binary arithmetic - Subtraction

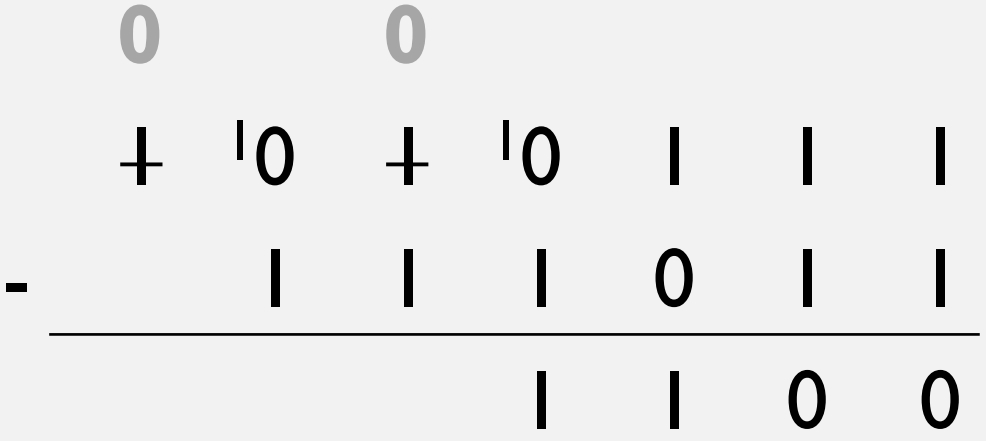


Binary arithmetic - Subtraction

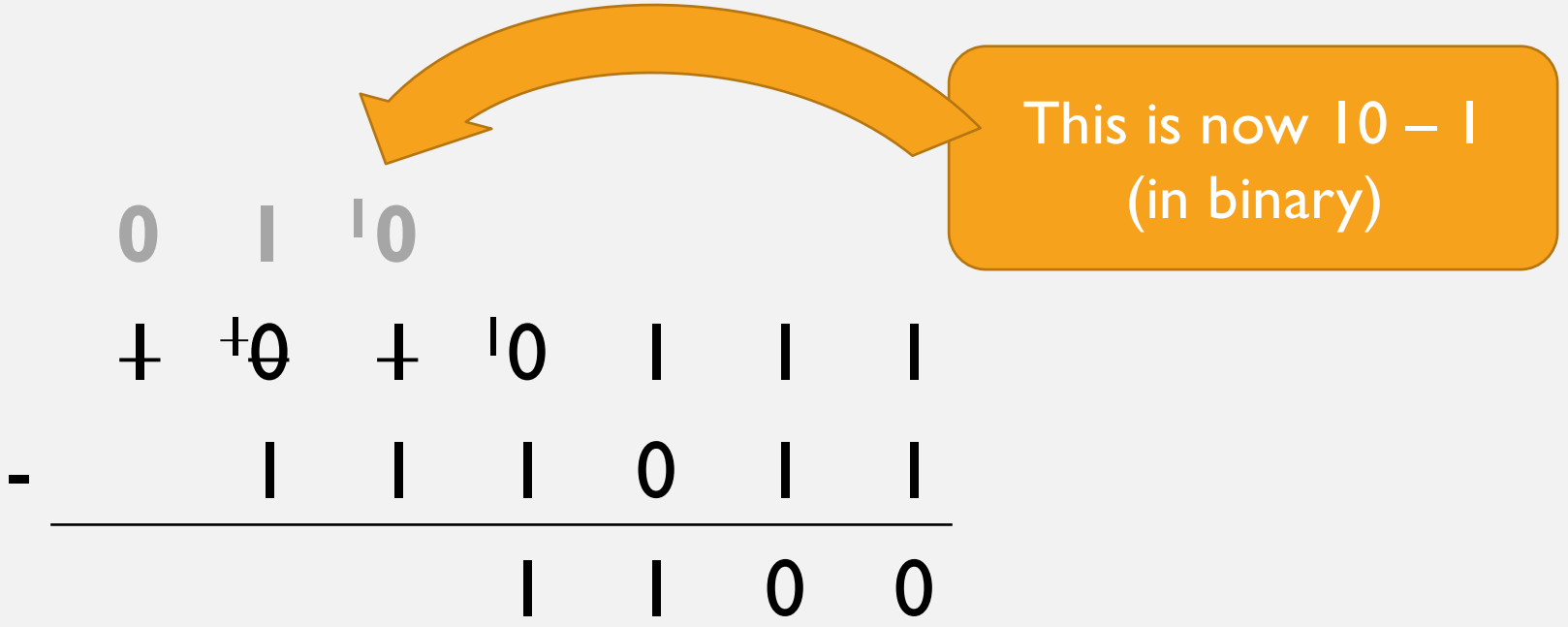


Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.

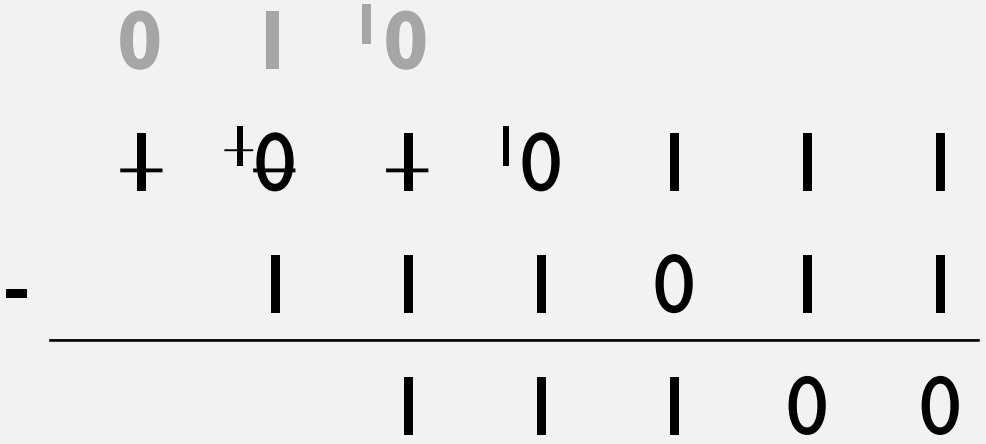


Binary arithmetic - Subtraction



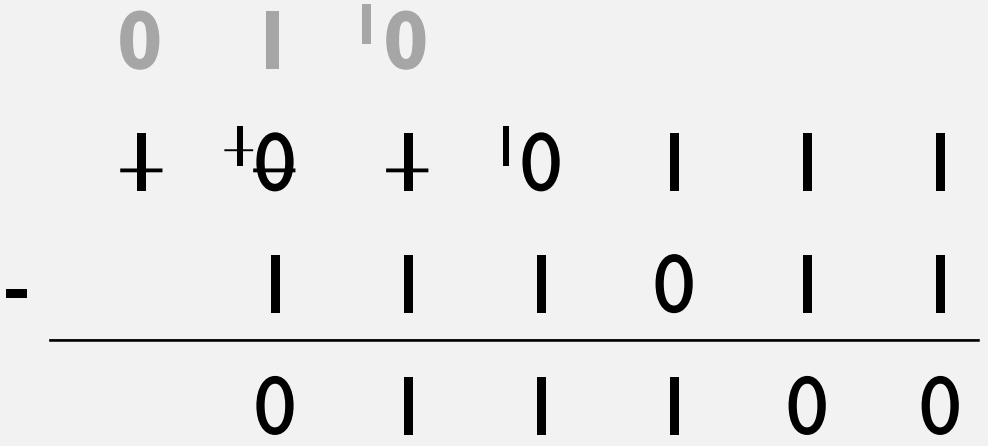
Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.



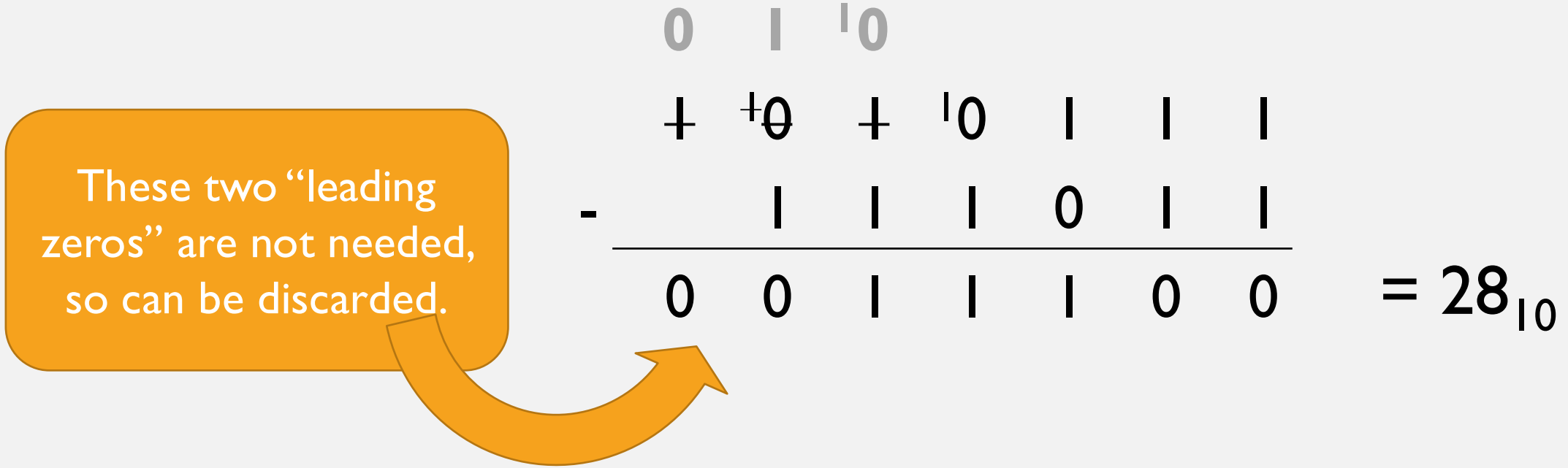
Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.



Binary arithmetic - Subtraction

For subtraction we have a concept of "**borrowing**" from a higher position.



Try it out

1 0 0 0 0 0 0
- 0 0 0 0 0 0 1

Try it out

```
  1 0 0 0 0 0 0
- 0 0 0 0 0 0 1
```

Look left for something to borrow. The next column is 0. And the next. All zeros until the leftmost column. So the borrow has to travel the whole way

Try it out

$$\begin{array}{r} \cancel{1}\,\cancel{0}\,\cancel{0}\,\cancel{0}\,\cancel{0}\,\cancel{0}\,0 \\ -\;\;0\;0\;0\;0\;0\;0\;1 \end{array}$$

Take the 1 from the leftmost column. It becomes 0, and the column to its right receives 2 (written as 10_2).

Try it out

$$\begin{array}{r} 0^{1}0\ 0\ 0\ 0\ 0\ 0 \\ -\ 0\ 0\ 0\ 0\ 0\ 0\ 1 \\ \hline \end{array}$$

Take the 1 from the leftmost column. It becomes 0, and the column to its right receives 2 (written as 10_2).

Try it out

$$\begin{array}{r} 0^{1}0^{1}0^{1}0^{1}0^{1}0^{1}0 \\ -\ \ 0\ 0\ 0\ 0\ \ 0\ 0\ 1 \end{array}$$

Take the 1 from the leftmost column. It becomes 0, and the column to its right receives 2 (written as 10_2). But that column also needs to lend 1 rightward - so it keeps 1 and passes 1 on. – all the way down the chain!

Try it out

$$
\begin{array}{r}
0^{1}0^{1}0^{1}0^{1}0^{1}0^{1}0 \\
-\ \ 0\ 0\ 0\ 0\ \ 0\ 0\ 1 \\
\hline
1
\end{array}
$$

Try it out

$$
\begin{array}{r}
0^{1}0^{1}0^{1}0^{1}0^{1}0^{1}0 \\
-\ \ 0\ 0\ 0\ 0\ \ 0\ 0\ 1 \\
\hline
0\ 1\ 1\ 1\ 1\ 1\ 1
\end{array}
$$

Binary Multiplication

Binary arithmetic - Multiplication

For multiplication, we multiply one of the numbers by each digit in the other number, and then sum them together.

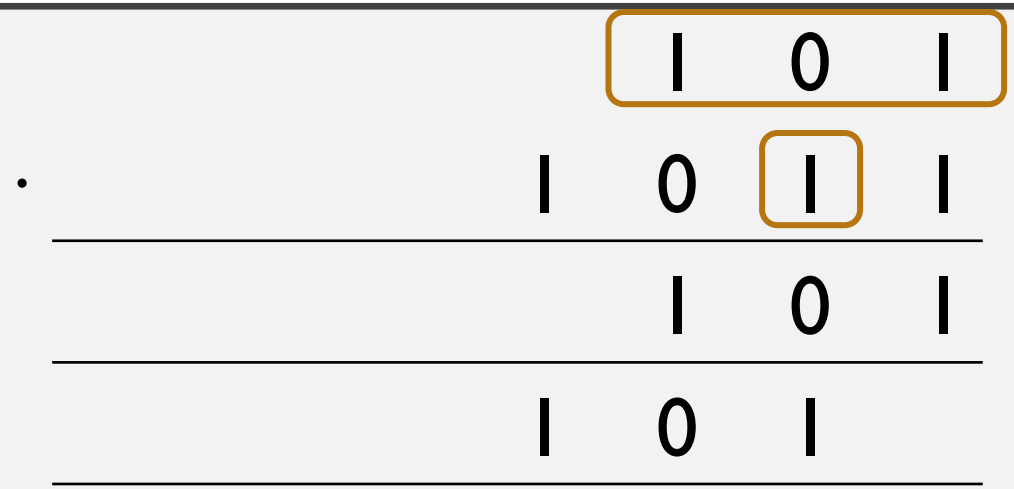
Binary arithmetic - Multiplication

$$
\begin{array}{r}
1\ 0\ 1 \\
\cdot\ 1\ 0\ 1\ 1 \\
\hline
\end{array}
$$

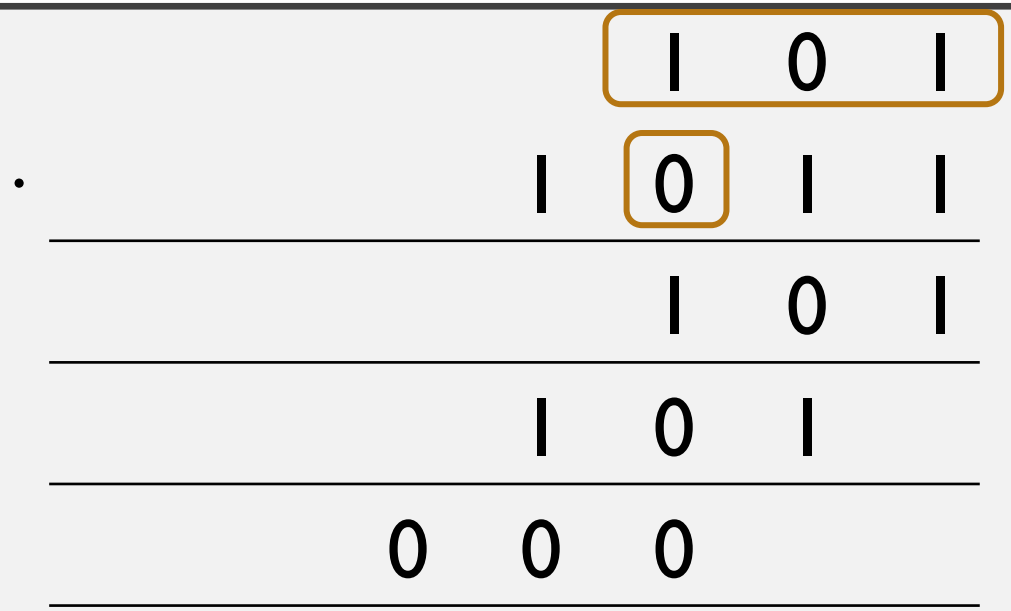
Binary arithmetic - Multiplication

$$
\begin{array}{r}
1\ 0\ 1 \\
\cdot\quad 1\ 0\ 1\ 1 \\
\hline
1\ 0\ 1 \\
\hline
\end{array}
$$

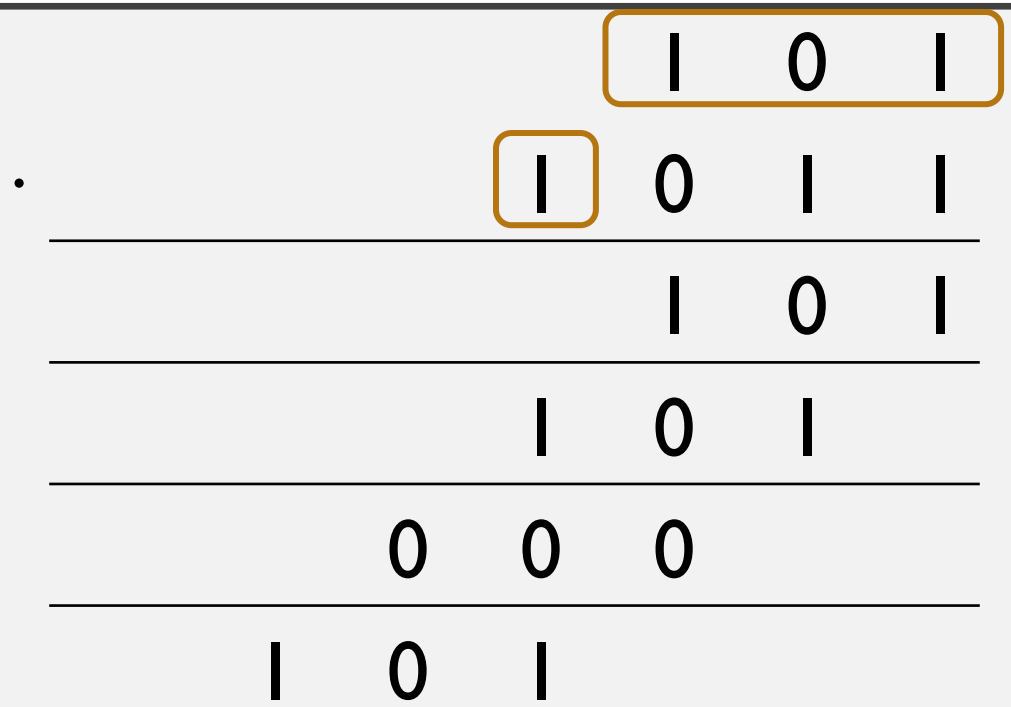
Binary arithmetic - Multiplication



Binary arithmetic - Multiplication

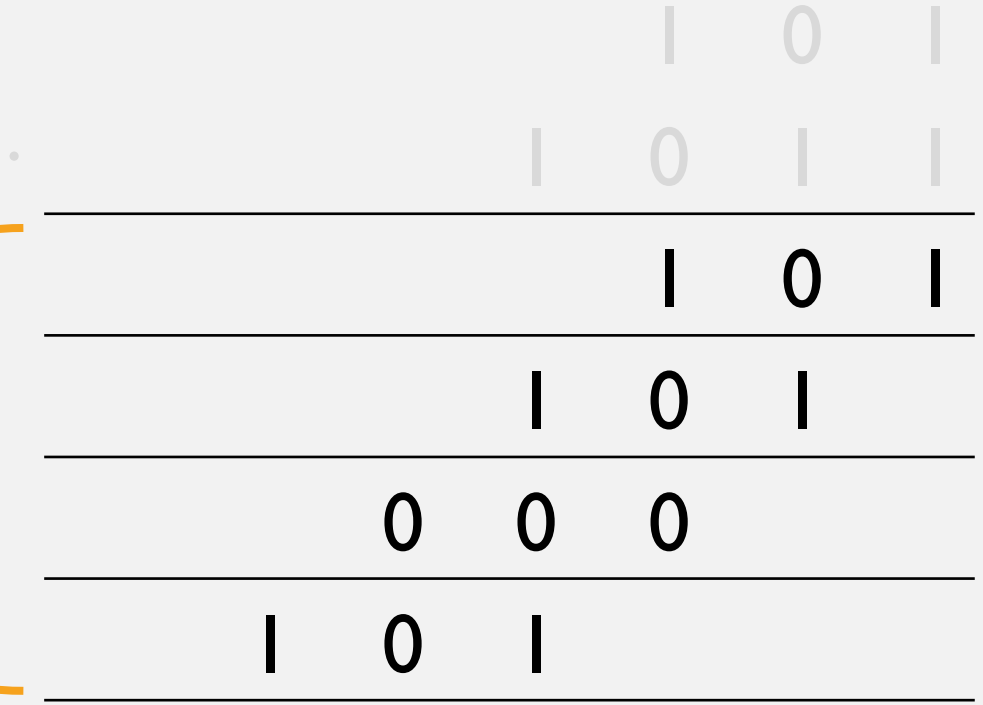


Binary arithmetic - Multiplication

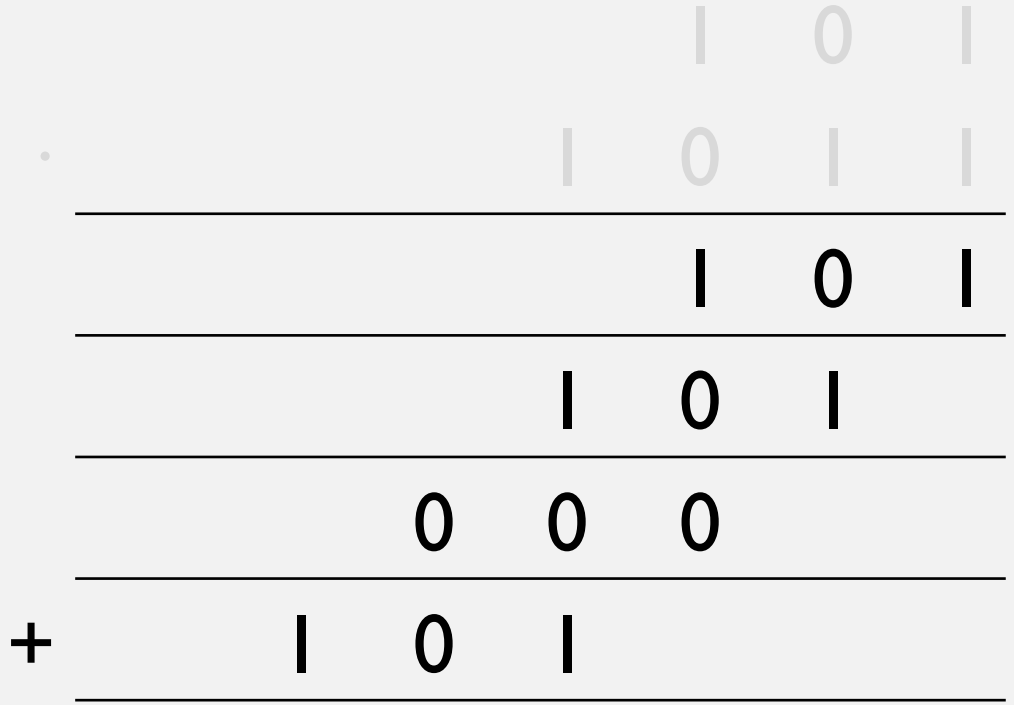


Binary arithmetic - Multiplication

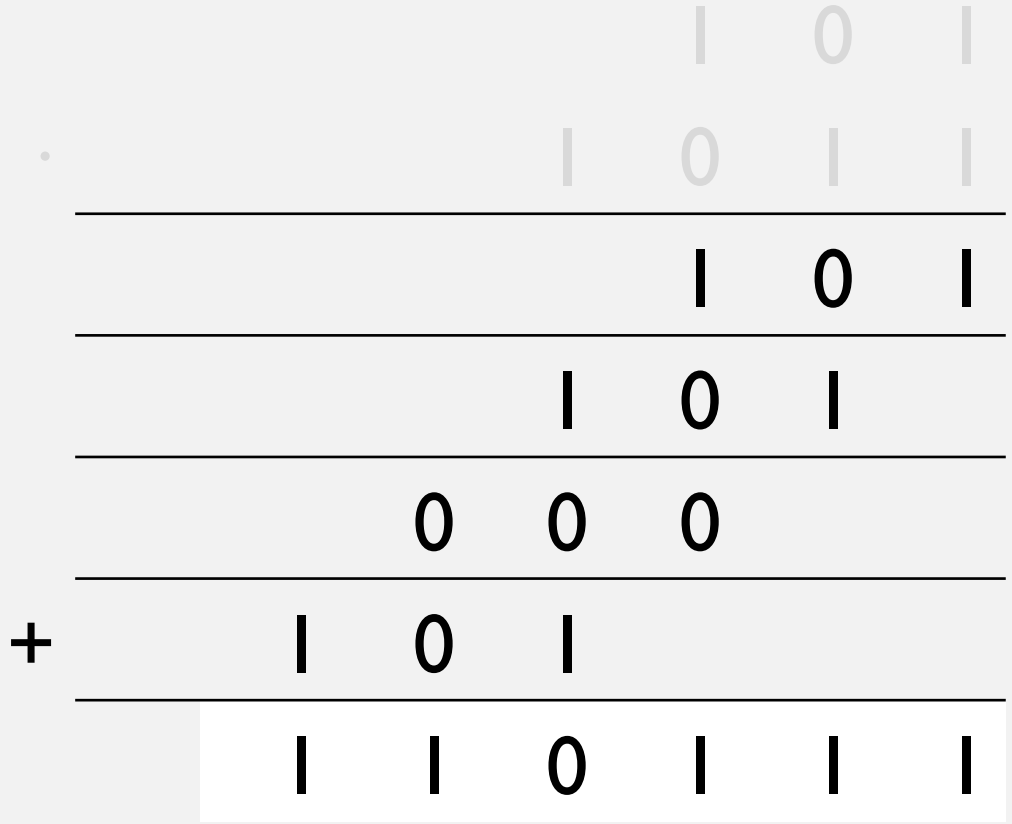
Sum up these intermediate values



Binary arithmetic - Multiplication



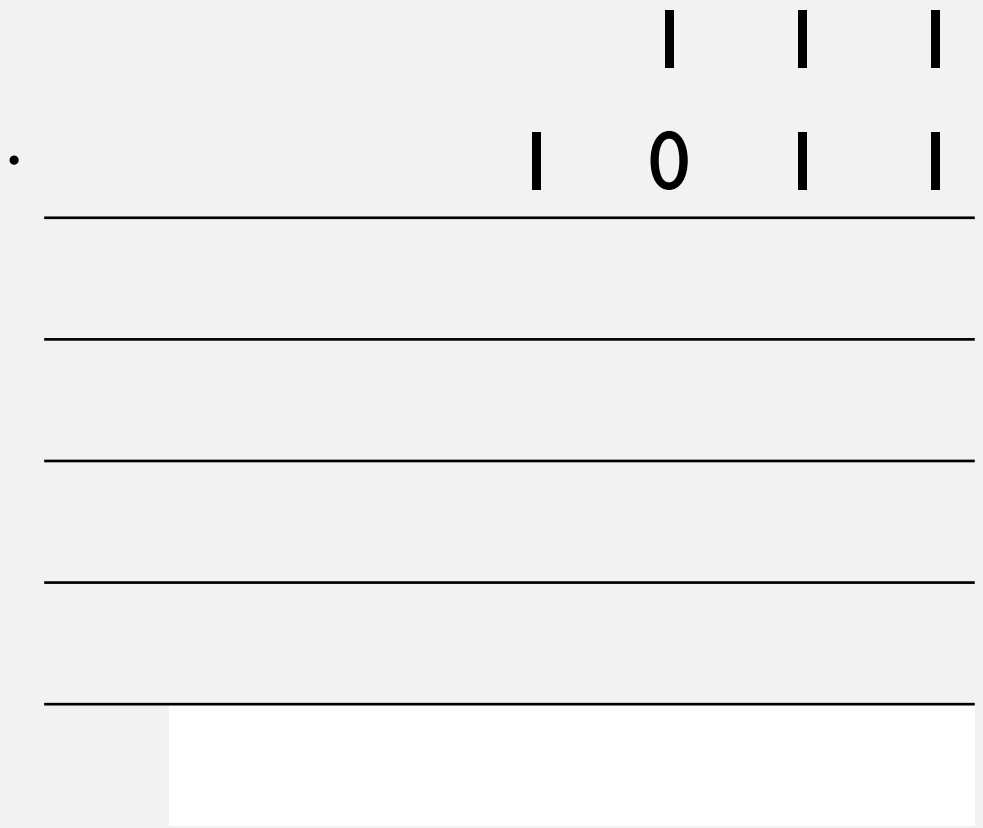
Binary arithmetic - Multiplication



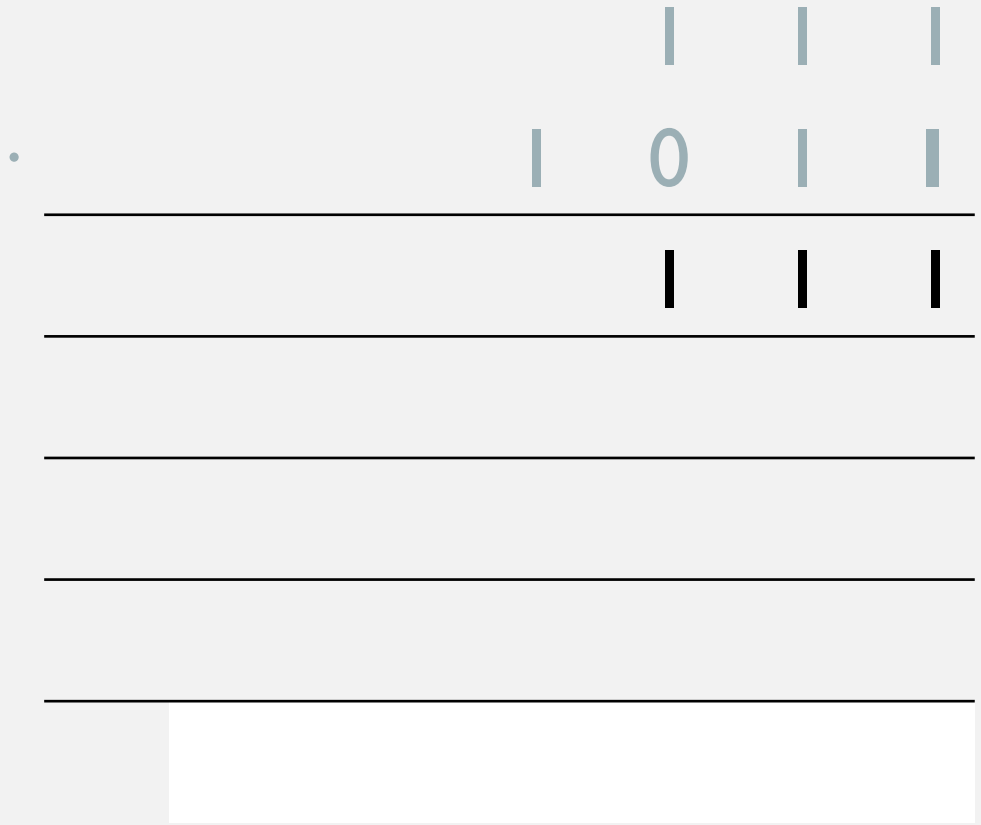
Try it out



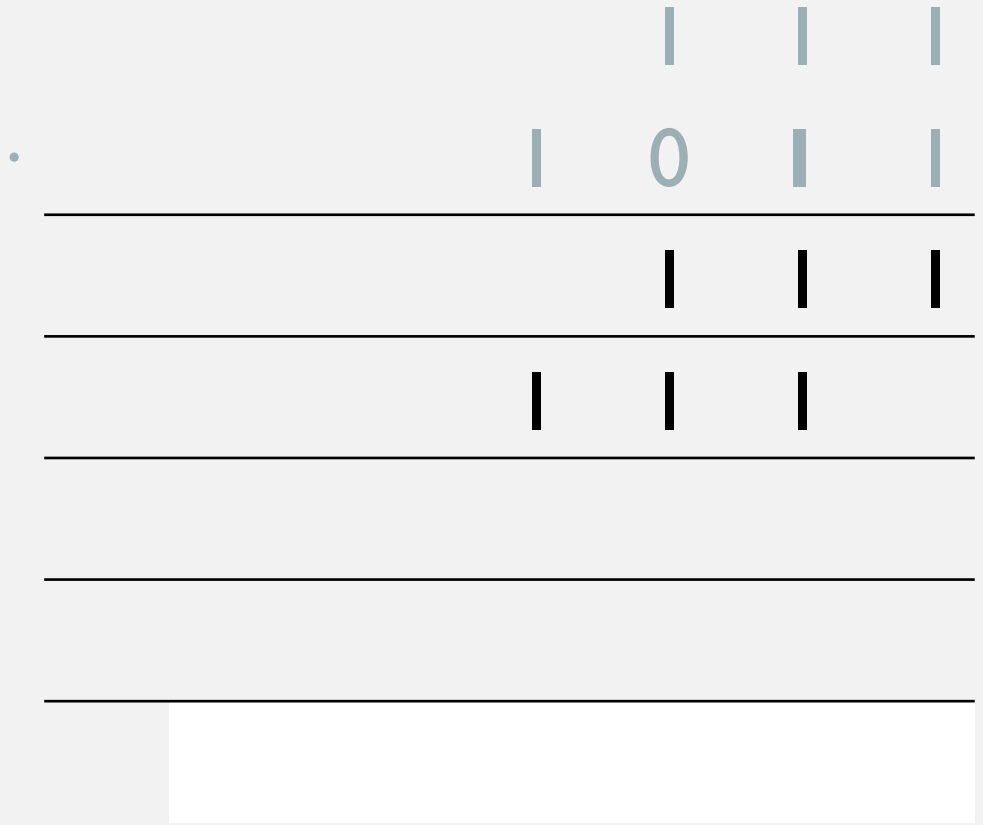
Binary arithmetic - Multiplication



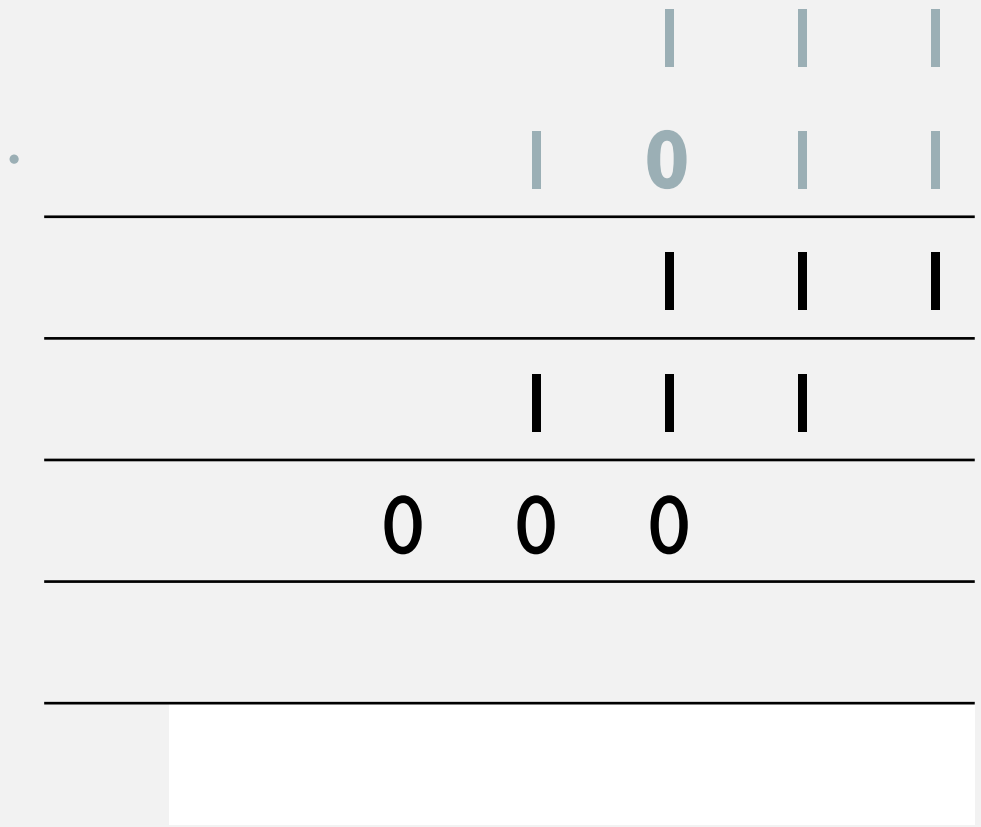
Binary arithmetic - Multiplication



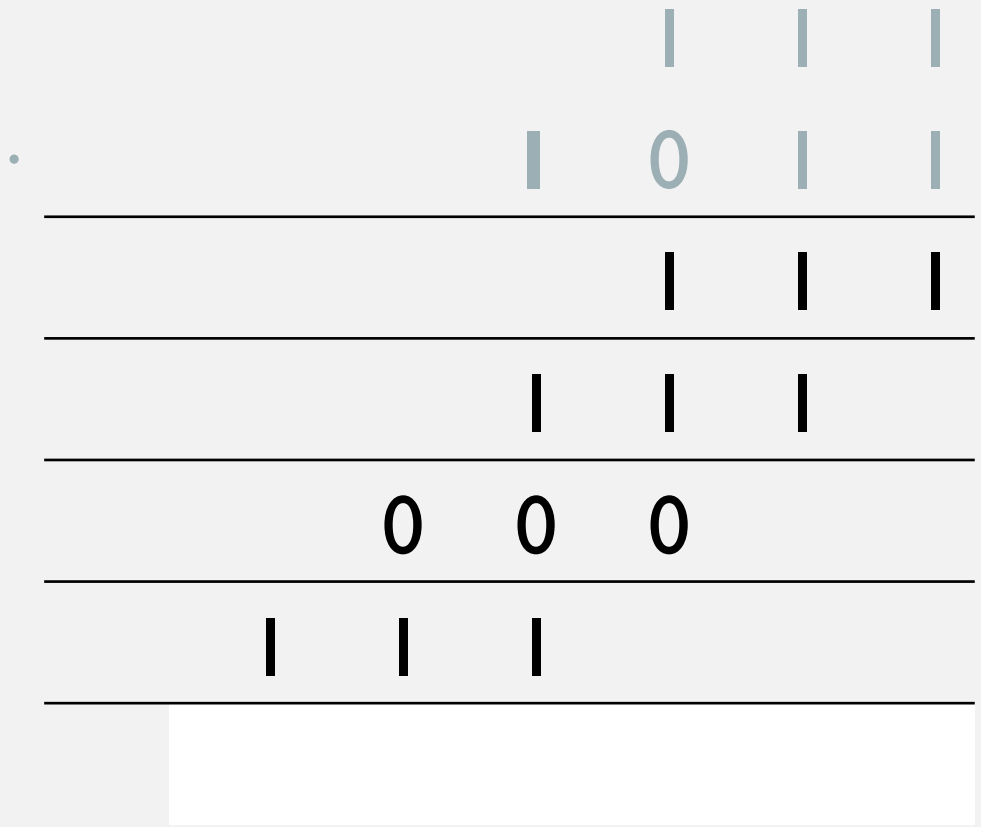
Binary arithmetic - Multiplication



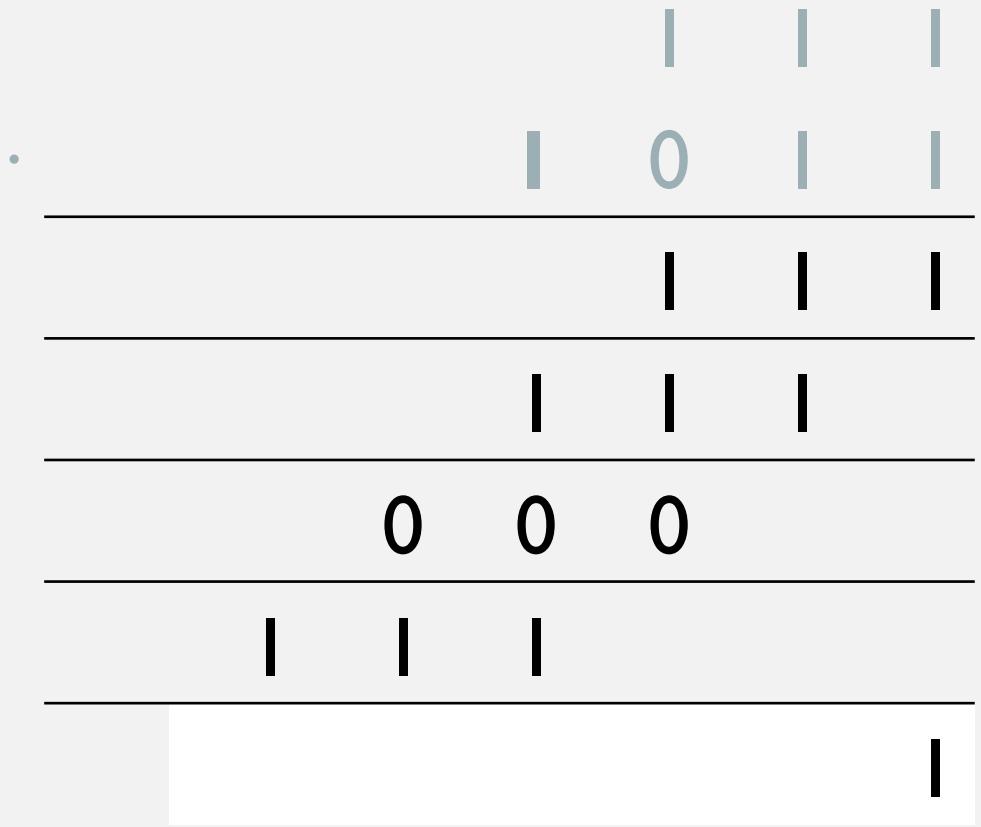
Binary arithmetic - Multiplication



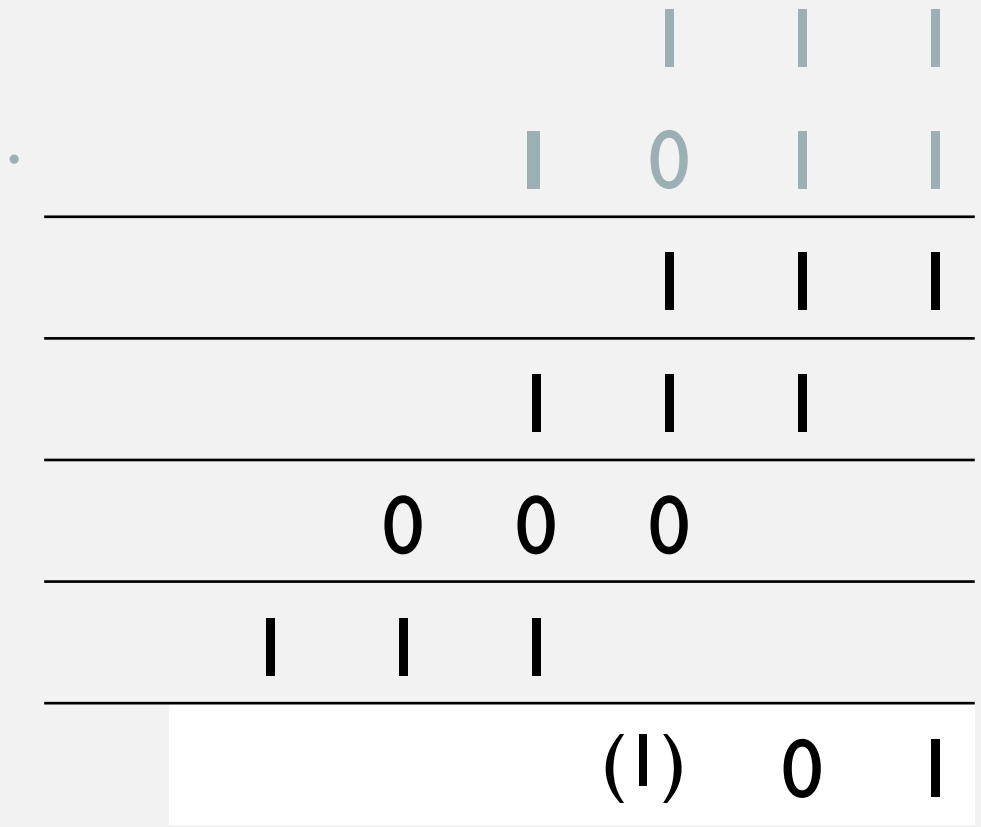
Binary arithmetic - Multiplication



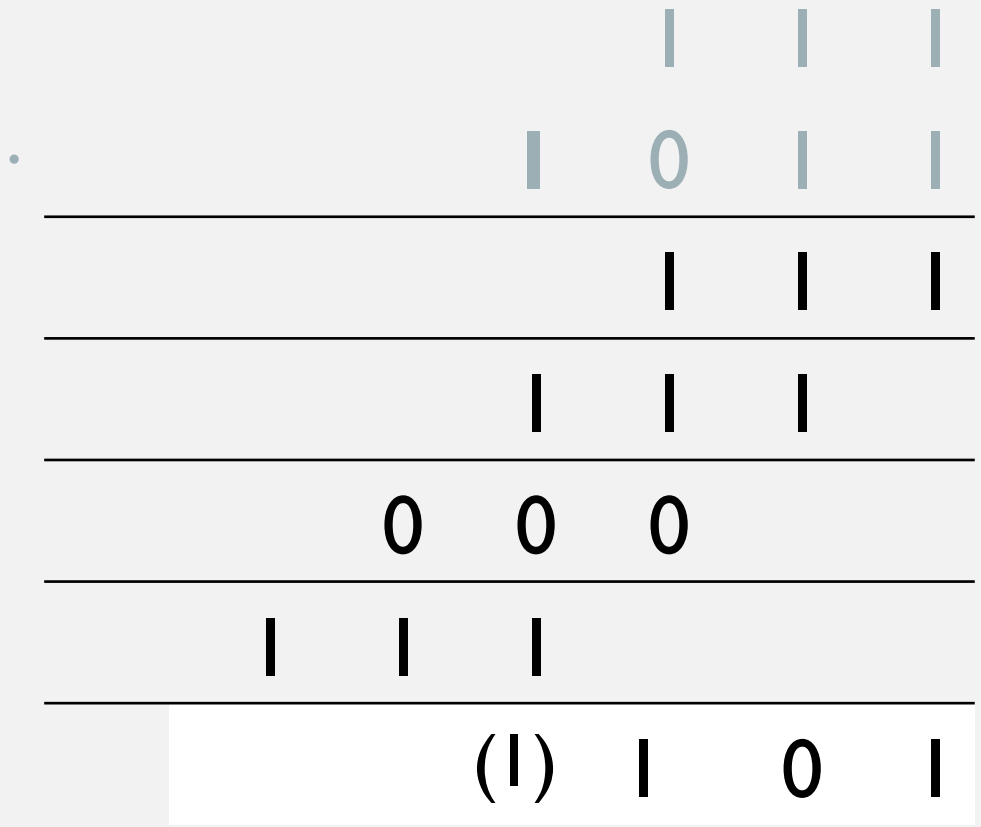
Binary arithmetic - Multiplication



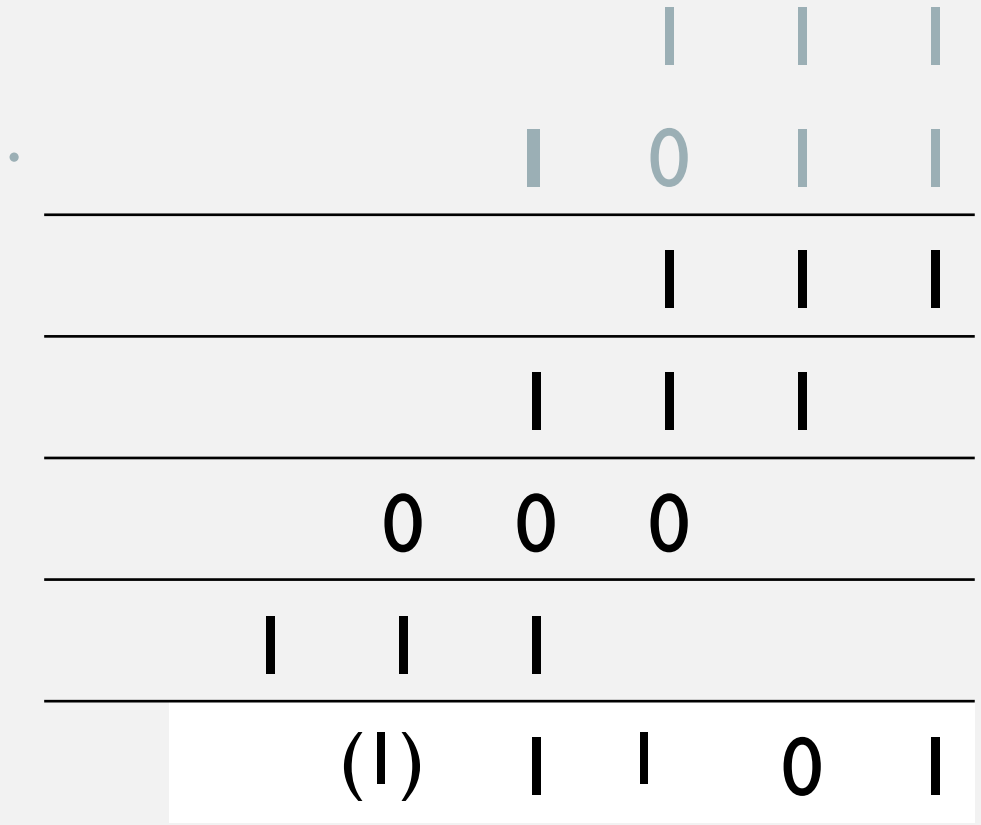
Binary arithmetic - Multiplication



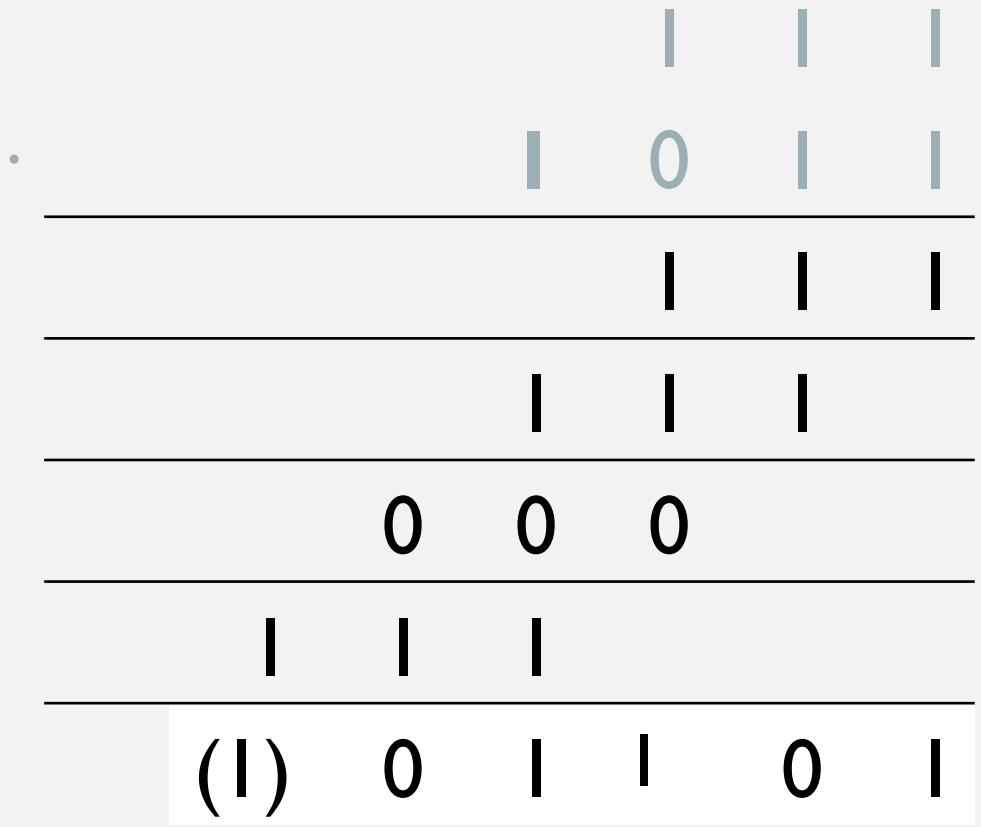
Binary arithmetic - Multiplication



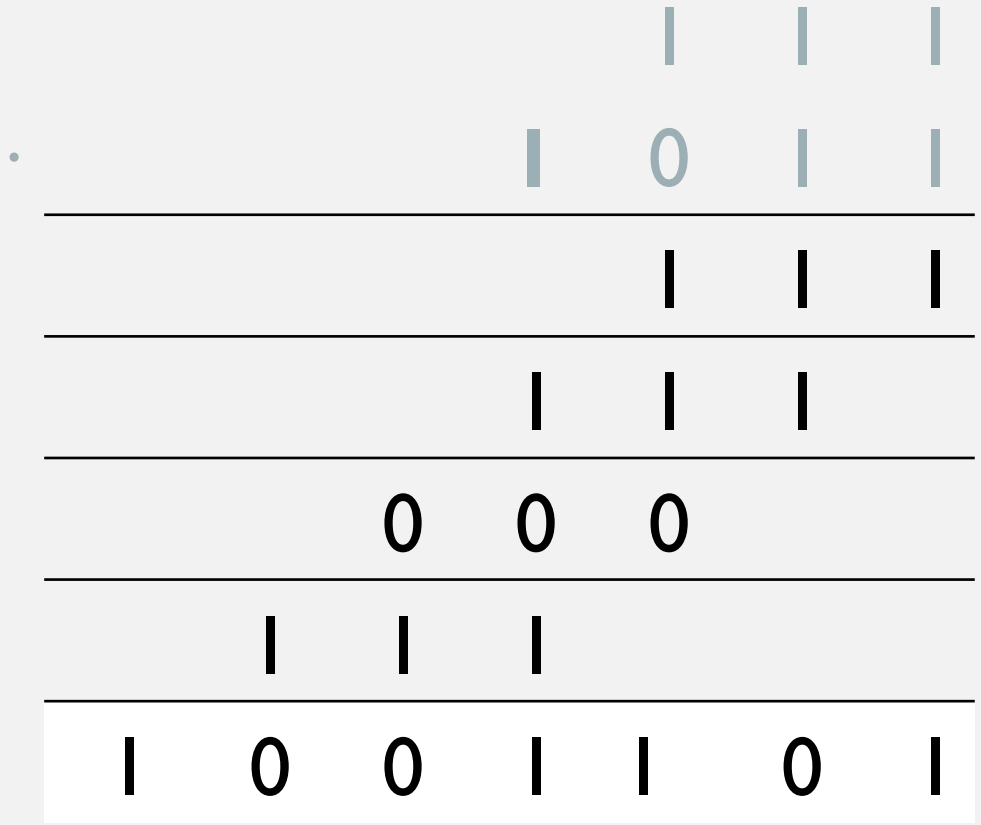
Binary arithmetic - Multiplication



Binary arithmetic - Multiplication



Binary arithmetic - Multiplication



Why does this matter?

Why do we want this?

- Development of computers as we know them today required the ability to **store and manipulate information**.
- Being able to **reliably** do this is key to the success of modern computers, and developing hardware to do this is tricky.
- It's much easier to differentiate **two** levels of electrical **voltage** (off vs on), which makes the **binary** system so attractive
- Often we treat low voltage as 0, and high voltage as 1.

Why do we want this?

- We call a single binary value a "**bit**" or "**b**inary dig**it**"
- We then group bits together into a "**byte**", which often refers to 8 bits
- We often use some multiple of 8 bits to represent data in our machine, and modern computers are designed to carry out instructions on this. We call this a "**word**". The "**word size**" of the computer defines the maximum number of bits the processor can operate on at a time.

Representation

- You may have noticed that all the numbers we have discussed so far have been Whole Numbers {0, 1, 2, 3, …}
- What about …
  - Negative numbers
  - Floating Point numbers
- These have special properties (a sign, a fractional part etc.) which we would want to represent

Representation

- Data can be quite complex, but we can utilise binary bits to build up a representation of these complex things from the atomic 0s and 1s.
- As long as we can agree on how to convert a collection of 0s and 1s to our "thing", and vice-versa, then we can represent it on the machine and utilise it for computation.
- In the next session we will look at representation of data.

Why do we want this?

- We can go even further than representing the categories of numbers discussed earlier.

- We will use numbers to represent many different things:
  - Characters
  - Colors
  - Shapes
  - Objects
  - …

Have a think between now and then. How would you represent these things if all you have is binary?

Goals for Today

- Categories of numbers and positional notation
- Overview of bases 2, 8, and 16
- Converting numbers between bases

CS-107 Lecture 5
Two's Complement

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- Data on a finite machine
- Representation of whole numbers
- Representation and arithmetic on negative numbers

Finite vs. Infinite

- A computer can only process a finite amount of data in finite time.
- How do we represent an infinite world and compute on that representation?
- We have to represent enough information to satisfy our goals.
- Too little information and the result may not be good enough. Too much information and the computation may be too expensive.

Analog vs. Digital

- We often need to **digitise** our **analog data**
- Sometimes this is okay (converting integers for example), but sometimes this leads to an approximation of the original:



Analog Signal

Digital Signal

Binary Representations

- If **one** bit can represent 2 things (0,1)…
- **Two** bits can represent 4 things (00,01,10,11)…
- **Three** bits can represent…? (000, 001, 010, 100,…, 111)
- **Four** bits? **Eight** bits? **Sixty-four** bits?

- For each extra bit, the number of things we can represent **doubles**. With n bits, we can represent 2^n different things…

How many *things*

n	**2^n**	
1	2	Two
2	4	Four
3	8	Eight
4	16	Sixteen
8	256	Two Hundred and Fifty Six
16	65536	~ Sixty Five Thousand
32	4,294,967,296	~ Four Billion
64	18446744073709551616	~ Eighteen Quintillion

Representing the Natural Numbers

- **Simple**: Interpret the bit pattern as a binary number.
- 4 bits lets us represent the numbers 0-15:

Bit Pattern	Number	Bit Pattern	Number
0000	0	1000	8
0001	1	1001	9
0010	2	1010	10
0011	3	1011	11
0100	4	1100	12
0101	5	1101	13
0110	6	1110	14
0111	7	1111	15

Representing Negative Numbers

- First idea: Let's use the **first bit** to denote whether the number is positive (0) or negative (1)!

- This is called Sign-Magnitude representation.
  - You have a sign bit (the first bit)
  - The remaining bits denote the magnitude (or value)

Why might this be a problem?

Sign-Magnitude Representation

- First idea: Let's use the **first bit** to denote whether the number is positive (0) or negative (1)!

Bit Pattern	Number	Bit Pattern	Number
0000	0	**1**000	-0
0001	1	**1**001	-1
0010	2	**1**010	-2
0011	3	**1**011	-3
0100	4	**1**100	-4
0101	5	**1**101	-5
0110	6	**1**110	-6
0111	7	**1**111	-7

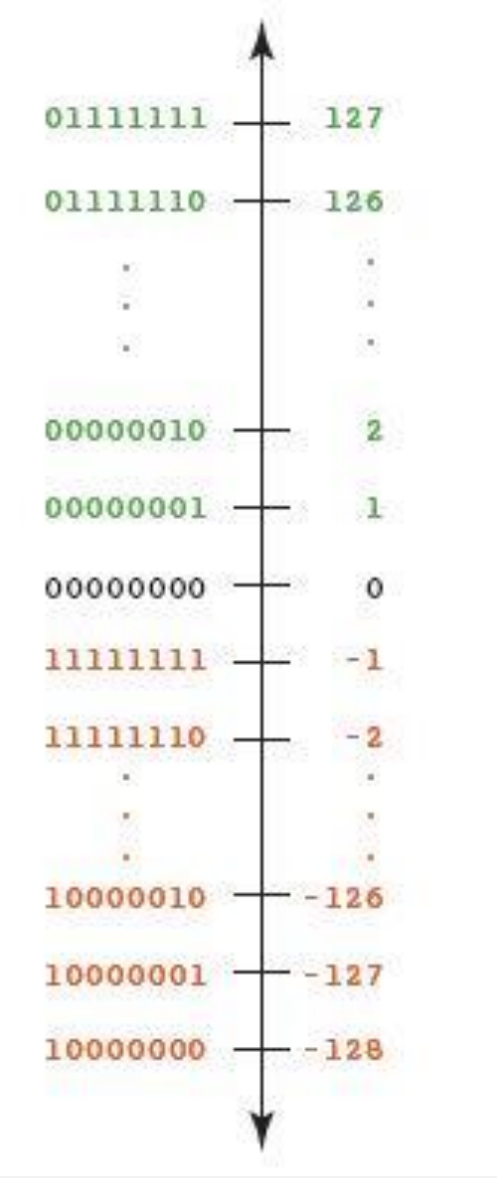
Representing Negative Numbers

- Revised idea: We'll still have the first bit denote positive and negative, but we will let the numbers **wrap around**.
- Start counting from 0, but when we use up the *n-1* positions we resume counting from **greatest magnitude negative number**.
- Known as Two's Complement, or Modulo Arithmetic

Two's Complement

- Revised idea: We'll still have the first bit denote positive and negative, but we will let the numbers **wrap around**.

Bit Pattern	Number	Bit Pattern	Number
0000	0	**1**000	-8
0001	1	**1**001	-7
0010	2	**1**010	-6
0011	3	**1**011	-5
0100	4	**1**100	-4
0101	5	**1**101	-3
0110	6	**1**110	-2
0111	7	**1**111	-1



Two's Complement:

A mapping where the negative values are mapped to the upper half of the possibly representable numbers.

To calculate the negative value of a value:

Negative(x) = $2^k - x$

(in binary we can just use flip and +1)

Ranges with ints of different sizes

- In computing, a range limited (i.e. fixed bit-length) integer is known as an **int**.
- Depending on how it is used, we can have **signed** and **unsigned** ints.

	Range	
Number of Bits	**Unsigned int**	**(signed) int**
8 bits	0 – 255	-128 – 127
16 bits	0 – 65535	-32768 – 32767
32 bits	0 – 4294967295	-2147483648 – 2147483647
64 bits	0 – 18446744073709551615	-9223372036854775808 – 9223372036854775807

Negating a Two's Complement value

- To get the negative of a number in Two's Complement, we:
  - Invert the bits (i.e. swap 1s for 0s and vice-versa)
  - Add 1

E.g. Negate 5 in 4-bit Two's Complement	
5_{10} =	0101_2
Invert the bits =	1010_2
Add 1 to the result =	1011_2
-5_{10} =	1011_2

E.g. Negate -2 in 4-bit Two's Complement	
-2_{10} =	1110_2
Invert the bits =	0001_2
Add 1 to the result =	0010_2
2_{10} =	0010_2

Arithmetic with Two's Complement

- Addition: add the numbers which represent the number of interest, and then throw away any carried digits beyond.

	2
+	3
	5



	0	0	1	0
+	0	0	1	1
	0	1	0	1

Arithmetic with Two's Complement

- Addition: add the numbers which represent the number of interest, and then throw away any carried digits beyond.

	5
+	-6
	-1



	0	1	0	1
+	1	0	1	0
	1	1	1	1

Arithm mplement

Bit Pattern	Number	Bit Pattern	Number
0000	0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

- Addition: add e number of interest, and t s beyond.

	5
+	-6
	-1



	0	1	0	1
+	1	0	1	0
	1	1	1	1

Arithmetic with Two's Complement

- Addition: add the numbers which represent the number of interest, and then throw away any carried digits beyond.

	-3
+	-2
	-5



+	1			
	1	1	0	1
+	1	1	1	0
	1	0	1	1

Arithm mplement

Bit Pattern	Number	Bit Pattern	Number
0000	0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

- Addition: add e number of interest, and t s beyond.

	-3
+	-2
	-5



	1	1	0	1
+	1	1	1	0
	1	0	1	1

Arithmetic with Two's Complement

- Subtraction: Use identity a – b = a + (-b)
- As we know how to negate a value, and we know how to apply addition, we can now do subtraction easily!

	2
-	3
	-1



	~~0~~	~~0~~	~~1~~	0
-	0	0	1	1

Nothing to borrow from!

Arithm m

Bit Pattern	Number	Bit Pattern	Number
0000	0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

Instead of -(+3) How about +(-3)

- Subtraction: U
- As we know h ow how to apply addition, we c

	2
-	3
	-1



	0	0	1	0
-	0	0	1	1



	0	0	1	0
+	1	1	0	1
	1	1	1	1

Arithmetic with Two's Complement

- Subtraction: Use identity a – b = a + (-b)
- As we know how to negate a value, and we know how to apply addition, we can now do subtraction easily!

	2
-	3
	-1



	0	0	1	0
-	0	0	1	1



	0	0	1	0
+	1	1	0	1
	1	1	1	1

Arithmetic with Two's Complement

- Subtraction: Use identity a – b = a + (-b)

Example of 2 – 3:

In 4-bit Two's Complement:	**0010 – 0011**
Negate the second number:	**0010 + 1101**
Add the two together:	**1111**
Result is:	**-1**

Integer Overflow

- So far we've only discussed calculations where the result is within the range of numbers that can be represented with our given scheme.
- What happens if we go beyond this?
  - If 4-bit Two's Complement can represent the range -8 to 7, what happens if we do 5+5? -5 – 5?
- The result exceeds the representable range of numbers.

Integer Overflow

- Example: 5 + 5 in 4-bit Two's Complement:

	1		1	
	0	1	0	1
+	0	1	0	1
	1	0	1	0

- In our 4-bit Two's Complement scheme, the result here is -6, not 10!

Integer Overflow

- Example: -5 - 5 in 4-bit Two's Complement:
- Change this to -5 + -5:

+		1	1	
	1	0	1	1
+	1	0	1	1
	0	1	1	0

- In our 4-bit Two's Complement scheme, the result here is 6, not -10!

Integer Overflow

- The problem here is that the value has **overflowed** beyond our representable range.
- It doesn't matter if the value goes too high, or too negative, both result in "overflow".
- Programmers must ensure that the result is in range, or they must handle an overflow error in some other way

Integer Overflow in the Real World

Ariane 5, 1996

- **4 June 1996:** flight 501 broke up 37 seconds after launch, destroying $370M of rocket and satellites.
- **The overflow:** a 64-bit floating-point velocity was written into a 16-bit signed integer, range -32768 to 32767. Nothing checked the fit.
- **Why it passed review:** the code came from Ariane 4, which never flew fast enough to exceed 32767.

It still happens

- **Exchange, 1 Jan 2022:** signature dates were held as YYMMDDhhmm in a signed 32-bit integer. 2201010001 exceeded 2147483647, and mail stopped flowing worldwide.
- **Boeing 787, 2015:** a generator counter overflowed after 248 days of continuous power. The interim fix was to reboot the aircraft.
- **Still ahead, 19 Jan 2038:** 32-bit Unix time runs out and wraps to a date in the past.

Sources: ESA/CNES Ariane 501 Inquiry Board report, 1996; Microsoft Exchange Team blog, January 2022; FAA airworthiness directive 2015-09-51.

Goals for Today

- Data on a finite machine
- Representation of whole numbers
- Representation and arithmetic on negative numbers

CS-107 Lecture 5
Two's Complement

Troy Astarte

Megan Venn-Wycherley